BUSINESS REQUIREMENTS DOCUMENT

Code-Sentinel

A Multi-Agent Code Review Platform Built on Service-Oriented Architecture

Submission Date - 22 August 2026

1. Introduction

1.1 Purpose of this Document

This Business Requirements Document (BRD) defines the functional and non-functional requirements, user stories, use cases, interface requirements, initial-draft wireframes and acceptance criteria for Code-Sentinel. It is written for the development team and any future contributor who needs a single reference for what the system must do before implementation of the System Design Document begins.

1.2 Project Overview

Code-Sentinel is a multi-agent code review platform that divides review work among five independent agent services — Security, Style, Performance, Logic and Documentation. A central orchestrator, built using LangGraph, calls all five services in parallel and merges their output into a single combined report. The platform is reachable from two client surfaces: a GitHub App that reviews pull requests automatically, and a Next.js web dashboard for configuration, history, reporting and on-demand re-runs. Style and lint issues are fixed automatically using deterministic tools (ESLint, Prettier, Ruff, Black); security, logic and performance issues are surfaced as AI-generated suggestions that the developer explicitly accepts or rejects.

1.3 Business Objectives and Success Metrics

Beyond the technical NFR targets in Section 4, Code-Sentinel is being built to move specific business/team-level outcomes. These objectives are what the project is ultimately judged against; the success metric for each is measured separately from — and is a leading indicator for — the underlying NFR.

Business Objective	Success Metric
Reduce the time a pull request waits for a first review comment, by resolving routine style/formatting feedback before a human ever looks at it.	≥70% reduction in style/formatting review comments on pilot repositories, measured before vs. after adoption.
Make first-level review consistent and available for every pull request, regardless of reviewer availability.	100% of pull requests on an enabled repository receive an automated Check Run within the MVP milestone.
Catch security and logic issues before merge instead of after release, when they are far more costly to fix.	At least 1 high-confidence security or logic finding surfaced per 20 PRs reviewed, demonstrated against the mid-semester demo dataset.
Give solo developers, students and open-source maintainers a working second opinion on code that would otherwise be merged unreviewed.	100% of PRs opened by first-time contributors on a pilot open-source repository receive an automatic review.
Demonstrate that the platform's service-oriented decomposition delivers real reusability, not just a theoretical architecture.	Each of the 5 agent services passes its own standalone REST/JSON contract test suite, callable independently of the Orchestrator.

1.4 Scope

In scope for the current academic project cycle:

- Automatic review of GitHub pull requests via webhook (Security, Style, Performance, Logic, Documentation agents).
- On-demand re-runs of a pull request's review from the web dashboard.
- A web dashboard for review history, combined report detail, and per-repository agent configuration.

- Automatic style/lint fixes for JavaScript/TypeScript and Python.
- AI-suggested fixes for security, logic and performance issues, posted as GitHub suggested changes.
- Context-aware review using a vector database of previously flagged issues.

Out of scope for the current cycle:

- Programming languages other than JavaScript/TypeScript and Python.
- IDE extensions.
- Automatic merging of pull requests.
- A dedicated native mobile application (the dashboard is mobile-responsive instead).

1.5 Definitions and Acronyms

Term	Meaning
BRD	Business Requirements Document (this document).
MVP	Minimum Viable Product — the smallest version of Code-Sentinel that is complete enough to demo, targeted for Week 10 of the project timeline.
SOA	Service-Oriented Architecture — the system is decomposed into independently deployable services.
SAST	Static Application Security Testing — scanning source code for vulnerabilities without executing it.
PR	Pull Request — a GitHub proposal to merge one branch into another.
API	Application Programming Interface — the contract client and service code use to communicate.
REST/JSON	Representational State Transfer over HTTP, with JSON (JavaScript Object Notation) as the request/response payload format — the interface style used by every service in Code-Sentinel.
CI	Continuous Integration — automated build/test pipelines that run on every code change.
LLM	Large Language Model, used by the Security, Logic, Performance and Documentation agents for AI-assisted analysis.
Orchestrator	The LangGraph-based service that fans a review request out to the five agents and aggregates their results.
Confidence threshold	The minimum confidence score an AI finding must have before it is posted as an auto-suggested fix. Defaults to 80% ("High") platform-wide; see FR-ORC-06 and FR-WEB-03.
Vector DB	Vector database (Pinecone / ChromaDB) used to retrieve semantically similar past issues.

2. Stakeholders and User Personas

Code-Sentinel serves five primary personas. Functional requirements, user stories and acceptance criteria in the sections that follow are traced back to these personas.

Persona	Description	Primary Goal
PR Author / Developer	Any team member opening a pull request, or re-running a review from the dashboard.	Get fast, actionable feedback at the point of work — the pull request — without waiting on a human reviewer.
Tech Lead / Reviewer	Senior engineer responsible for reviewing pull requests before merge.	Have routine issues (naming, formatting, missing null checks) resolved automatically, so review time goes to design and business logic.
Repository Admin	Installs the GitHub App and configures CodeSentinel for one or more repositories.	Control which agents run, set thresholds for auto-suggested fixes, and see review activity across all installed repositories.
Open-Source Maintainer	Maintains a public repository that receives contributions from unknown or first-time contributors.	Get a consistent first-level check on every external contribution before spending personal review time on it.
Solo Developer / Student	Works alone or in a small team with no dedicated reviewer available.	Have CodeSentinel act as a second opinion so code doesn't get merged completely unreviewed.

3. Functional Requirements

Requirements are grouped by service/module, mirroring the system's service-oriented decomposition. Priority follows a simplified MoSCoW scale: Must (required for the MVP milestone and mid-semester demo), Should (targeted for Phase 2/3), Could (stretch, time-permitting).

API Gateway

ID	Requirement	Priority
FR-GW-01	The Gateway shall be the single REST/JSON entry point (Express + TypeScript) for the GitHub App and Web Dashboard clients.	**Must**
FR-GW-02	The Gateway shall authenticate requests using GitHub App installation tokens for webhook/PR traffic and session authentication for Dashboard traffic.	**Must**
FR-GW-03	The Gateway shall verify the HMAC signature of every inbound GitHub webhook payload before it is accepted or forwarded.	**Must**
FR-GW-04	The Gateway shall route validated review requests to the Orchestrator and relay the Orchestrator's response back to the originating client.	**Must**
FR-GW-05	The Gateway shall rate-limit incoming requests per repository/installation to protect the downstream agent services from overload.	**Should**

Orchestrator (LangGraph)

ID	Requirement	Priority
FR-ORC-01	The Orchestrator shall fan out a review request to the five agent services (Security, Style, Performance, Logic, Documentation) in parallel over REST/JSON.	**Must**
FR-ORC-02	The Orchestrator shall apply a configurable per-agent timeout and return the partial set of results from agents that responded in time if one or more agents time out.	**Must**
FR-ORC-03	The Orchestrator shall aggregate all agent findings into a single combined review report and remove duplicate findings that reference the same file, line range and issue type.	**Must**
FR-ORC-04	The Orchestrator shall rank aggregated findings by severity (critical, warning, info) before returning the combined report to the Gateway.	**Must**
FR-ORC-05	The Orchestrator shall query the vector database for semantically similar past issues and attach relevant historical context to matching findings.	**Should**
FR-ORC-06	The Orchestrator shall generate a GitHub "suggested change" payload only for findings whose confidence score meets the repository's confidence threshold, which defaults to 80% ("High") platform-wide until a Repository Admin overrides it (FR-WEB-03). This Must-priority behavior does not depend on the override UI having shipped.	**Must**

Security Agent

ID	Requirement	Priority
FR-SEC-01	The Security Agent shall statically scan changed code (SAST) for common vulnerability patterns, including SQL injection, XSS, insecure deserialization and path traversal.	Must
FR-SEC-02	The Security Agent shall detect hardcoded secrets, API keys and credentials within changed lines of a diff.	Must
FR-SEC-03	The Security Agent shall return each finding with a severity level and a confidence score.	Must
FR-SEC-04	The Security Agent shall expose its findings through a documented REST/JSON API that can be called and reused independently of the rest of CodeSentinel.	Must

Style Agent

ID	Requirement	Priority
FR-STY-01	The Style Agent shall detect the language of each changed file and dispatch it to the corresponding linter/formatter (ESLint and Prettier for JavaScript/TypeScript; Ruff and Black for Python).	Must
FR-STY-02	The Style Agent shall execute all linters and formatters inside an isolated, resource-limited Docker sandbox per request.	Must
FR-STY-03	The Style Agent shall automatically apply safe style and formatting fixes without requiring an LLM call or developer confirmation.	Must
FR-STY-04	The Style Agent shall report any style issue it could not auto-fix as a normal finding in the combined report.	Should

Performance Agent

ID	Requirement	Priority
FR-PERF-01	The Performance Agent shall flag common performance anti-patterns in changed code, including N+1 query patterns, blocking calls on async/hot paths, and unnecessary loops or allocations.	Must
FR-PERF-02	The Performance Agent shall return its findings within the Orchestrator's configured per-agent timeout for typical diffs of under 500 changed lines.	Should

Logic Agent

ID	Requirement	Priority
FR-LOG-01	The Logic Agent shall analyze changed code for likely bugs and edge cases, including null/undefined handling, off-by-one errors and unhandled exceptions.	Must
FR-LOG-02	The Logic Agent shall use the shared LLM abstraction layer for analysis and automatically fall back to the next configured provider on failure, timeout or rate-limit.	Must

Documentation Agent

ID	Requirement	Priority
FR-DOC-01	The Documentation Agent shall check that public functions and classes touched by the diff include docstrings or comments describing parameters and return values.	**Must**
FR-DOC-02	The Documentation Agent shall suggest docstring content for missing or incomplete documentation as a non-blocking, informational finding.	**Should**

GitHub App

ID	Requirement	Priority
FR-GH-01	The GitHub App shall subscribe to pull_request webhook events (opened, synchronize, reopened) for every repository where it is installed.	**Must**
FR-GH-02	The GitHub App shall post the combined review report as a GitHub Check Run with a pass, fail or neutral conclusion.	**Must**
FR-GH-03	The GitHub App shall post high-confidence Security, Logic and Performance findings as inline pull request review comments containing a GitHub "suggested change".	**Must**
FR-GH-04	The GitHub App shall never modify the pull request branch directly; every AI-suggested change requires an explicit developer action ("Commit suggestion").	**Must**

Web Dashboard

ID	Requirement	Priority
FR-WEB-01	The Dashboard shall display review history across every repository the authenticated user's organization has installed CodeSentinel on.	**Must**
FR-WEB-02	The Dashboard shall let a Repository Admin enable or disable individual agents on a per-repository basis.	**Must**
FR-WEB-03	The Dashboard shall let a Repository Admin override the platform-wide default confidence threshold (80%, "High") used for auto-suggested fixes, on a per-repository basis. Until this is configured, FR-ORC-06's default applies.	**Should**

ID	Requirement	Priority
FR-WEB-04	The Dashboard shall display a combined review report detail view for a selected pull request, filterable by agent.	**Must**

Vector Database / Context

ID	Requirement	Priority
FR-VDB-01	The system shall store embeddings of previously flagged issues and their resolutions in the vector database.	**Could**
FR-VDB-02	The system shall retrieve semantically similar past issues for a new finding and surface them as supporting context in the report.	**Could**

4. Non-Functional Requirements

Non-functional requirements define how well the system must perform its functions — covering performance, scalability, availability, security, usability, maintainability, portability, observability, privacy and cost, consistent with the SOA principles the project is built to demonstrate.

ID	Category	Requirement	Target / Metric
NFR-01	Performance	End-to-end combined review for a typical pull request (under 500 changed lines) completes within 60 seconds.	P95 ≤ 60s
NFR-02	Performance	Per-agent timeout is configurable; default timeout is 20 seconds, after which the Orchestrator proceeds with partial results.	Default 20s timeout
NFR-03	Scalability	Each of the five agent services is independently deployable and horizontally scalable, consistent with SOA principles.	≥10 concurrent review jobs in demo environment
NFR-04	Availability / Reliability	If one agent service is unavailable or times out, the combined report is still produced using the remaining agents' results.	Graceful degradation, no full-pipeline failure
NFR-05	Security	All client-service and inter-service traffic uses HTTPS/TLS; GitHub webhook payloads are signature-verified; no secrets appear in logs or the database in plaintext.	TLS 1.2+, HMAC-verified webhooks
NFR-06	Security / Privacy	LLM providers receive only the diff / changed hunks required for analysis, not full source trees, to minimize exposure of proprietary code.	Diff-scoped LLM context only
NFR-07	Reliability	Style and lint auto-fixes are fully deterministic (rule/tool-based, no LLM) so they can be committed without developer review.	0 LLM calls in Style Agent auto-fix path
NFR-08	Usability	Severity levels, colors and terminology are consistent across the GitHub PR comments and Web Dashboard.	Shared design tokens across 3 clients
NFR-09	Usability	The Web Dashboard is responsive and remains usable down to a 375px-wide mobile viewport.	Mobile-responsive to 375px
NFR-10	Maintainability / Extensibility	Each agent service exposes a documented REST/JSON API contract and can be built, tested, deployed and reused independently of CodeSentinel.	Independent API contract per agent (SOA)
NFR-11	Portability	Agent services and the LLM abstraction layer run unchanged across local development (Ollama) and hosted environments (OpenRouter, Gemini Flash, Groq).	Provider-agnostic LLM interface
NFR-12	Observability	Every review request, agent latency, timeout and failure is logged and visible per run for troubleshooting.	Per-run latency & failure log
NFR-13	Data Privacy	Review data and code diffs are scoped and retained per installing GitHub organization only; no cross-tenant data sharing.	Strict per-tenant data isolation
NFR-14	Cost Control	LLM usage stays within provider free/low-cost tiers during development via the OpenRouter → Gemini Flash → Groq → Ollama fallback chain.	Zero/near-zero LLM spend in dev

5. User Stories and Use Cases

5.1 User Stories

Each user story is traced to the functional requirement(s) it realizes and the acceptance criterion that verifies it (Section 8). Priority mirrors the MoSCoW scale used for functional requirements (Section 3): Must-tier stories are required for the MVP milestone; Should-tier stories (US-08, US-10) target Phase 2/3 and depend on their linked Should-priority FRs shipping; the Could-tier story (US-13) is a stretch enhancement tied to the vector database and is not required for the MVP.

ID	Persona	Story	Pri.	Linked FR	AC
US-01	PR Author	As a PR author, I want CodeSentinel to automatically review my pull request when I open or update it, so that I get feedback without waiting for a human reviewer.	**Must**	FR-GH-01, FR-ORC-01	AC-01
US-02	PR Author	As a developer, I want to see security, style, performance, logic and documentation issues in one combined report, so that I don't have to check four different tools.	**Must**	FR-ORC-03, FR-ORC-04	AC-02
US-03	PR Author	As a developer, I want style and lint issues fixed automatically, so that I don't spend review time on formatting.	**Must**	FR-STY-01–03	AC-03
US-04	PR Author	As a developer, I want to review each AI-suggested fix and choose to accept or reject it, so that I stay in control of what changes in my code.	**Must**	FR-GH-03, FR-GH-04	AC-04
US-05	PR Author	As a developer, I want to re-run the review of an open pull request from the dashboard, so that I can get fresh results without a new commit.	**Must**	FR-WEB-01	AC-05
US-06	Tech Lead	As a tech lead, I want routine issues resolved before I look at a PR, so that I can spend my time on design and business logic instead of formatting.	**Must**	FR-STY-03, FR-ORC-04	AC-06
US-07	Repository Admin	As a repository admin, I want to choose which agents run on my repository, so that I only get the checks that are relevant to my project.	**Must**	FR-WEB-02	AC-07
US-08	Repository Admin	As a repository admin, I want to set the confidence threshold for auto-suggested fixes, so that I control how aggressive the AI suggestions are.	**Should**	FR-WEB-03, FR-ORC-06	AC-08
US-09	Repository Admin	As a repository admin, I want to see review history across all my repositories, so that I can track code quality trends over time.	**Must**	FR-WEB-01, FR-WEB-04	AC-09
US-10	OSS Maintainer	As an open-source maintainer, I want first-time contributors' pull requests automatically checked, so that I have a baseline before I review external code myself.	**Should**	FR-GH-01	AC-10

ID	Persona	Story	Pri.	Linked FR	AC
US-11	Solo Developer / Student	As a solo developer, I want CodeSentinel to act as my only reviewer, so that I still get a second opinion even though nobody is available to review my code.	**Must**	FR-GH-02, FR-ORC-03	AC-11
US-12	Any Developer	As a developer, if one of the five agents is slow or fails, I want to still get a report from the agents that succeeded, so that a single slow check doesn't block my whole review.	**Must**	FR-ORC-02	AC-12
US-13	PR Author	As a developer, I want the system to show me similar issues found in the past, so that I understand whether this is a recurring problem in the codebase.	**Could**	FR-VDB-01, FR-VDB-02	AC-13

5.2 Use Case Diagram — System Overview

The diagram below shows all primary actors and use cases for Code-Sentinel. Solid lines are actor-to-use-case associations; dashed arrows are <<include>>/<<extend>> relationships between use cases.

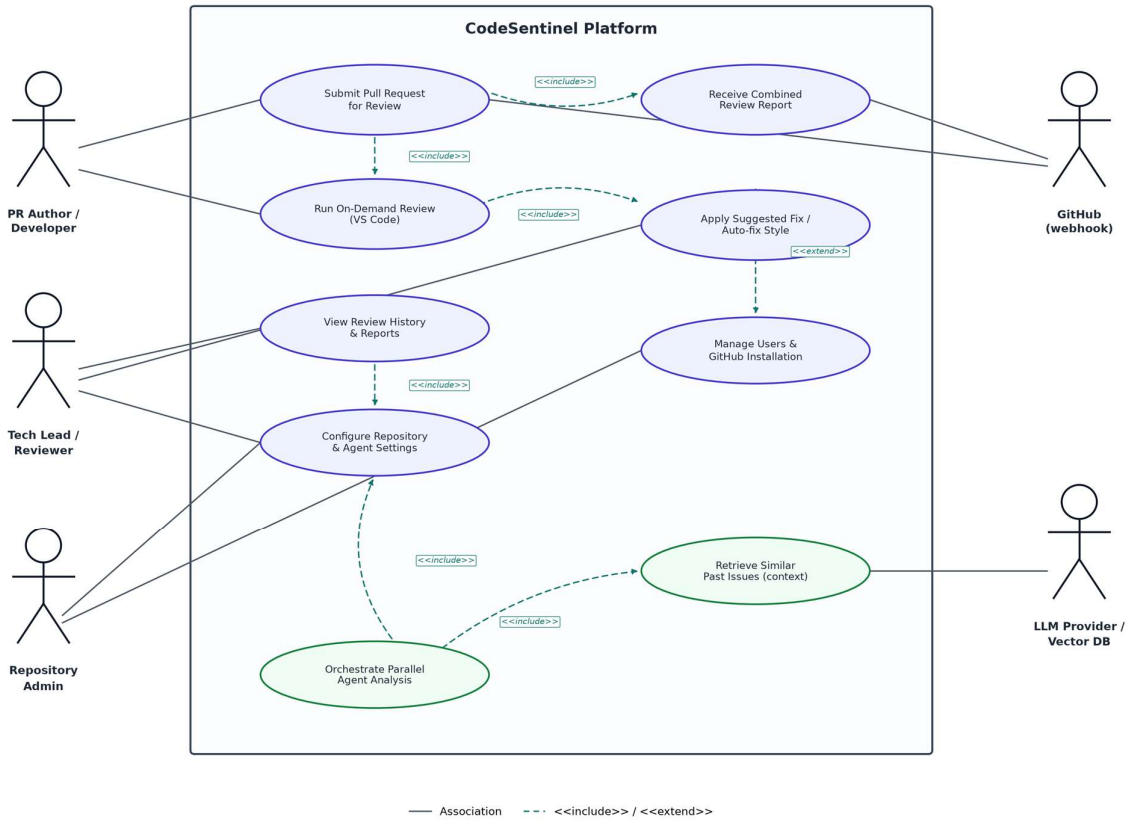


Figure 5.1 — Primary use case diagram for the Code-Sentinel platform

5.3 Detailed Flow — Automatic Pull Request Review

The activity diagram below expands UC-1 (Automatic Pull Request Review) across the five participating components, showing the fan-out to agent services and the fan-in during aggregation.

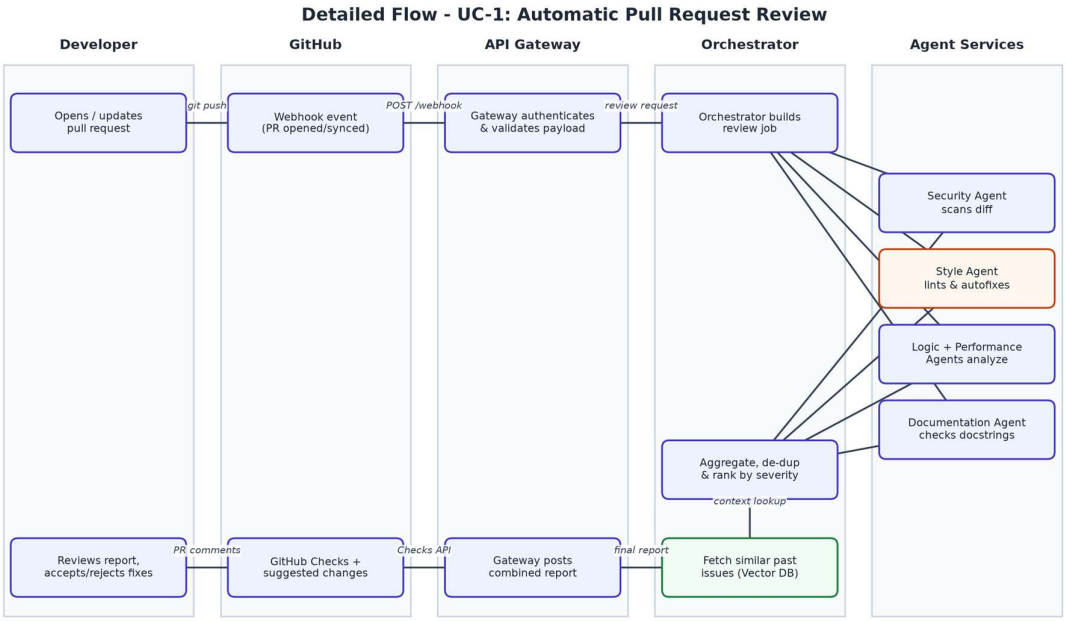


Figure 5.2 — Detailed swimlane flow for UC-1: Automatic Pull Request Review

5.4 Use Case Specifications

UC-1 — Automatic Pull Request Review

Field	Description
Actors	PR Author (primary), GitHub (webhook), Orchestrator, five Agent Services
Preconditions	The GitHub App is installed on the repository and the webhook is registered.
Main Flow	1. PR Author opens or pushes to a pull request. 2. GitHub sends a pull_request webhook event to the API Gateway. 3. The Gateway verifies the webhook signature and authenticates the request. 4. The Gateway forwards a review request to the Orchestrator. 5. The Orchestrator fans the request out to all enabled Agent Services in parallel. 6. Each Agent Service analyzes the diff and returns findings (or the Orchestrator's timeout elapses). 7. The Orchestrator aggregates, de-duplicates and severity-ranks the findings, and looks up similar past issues in the vector database. 8. The Gateway posts the combined report as a GitHub Check Run and posts high-confidence findings as inline suggested-change comments. 9. PR Author reviews the report and accepts or rejects each suggested change.
Alternate / Exception Flow	If an agent times out (step 6), the Orchestrator proceeds with the remaining agents' results and marks the timed-out agent as “no result” in the report.
Postconditions	A GitHub Check Run with a pass/fail/neutral conclusion exists on the pull request, and any accepted suggestions are committed to the PR branch.

UC-2 — On-Demand Review from the Dashboard

Field	Description
Actors	PR Author (primary), Web Dashboard, API Gateway, Orchestrator, Agent Services
Preconditions	The developer is signed in to the Web Dashboard and the pull request is open on GitHub.
Main Flow	1. Developer opens the repository's pull requests in the Web Dashboard and clicks “Re-run review” on an open pull request. 2. The Dashboard sends the repository and pull request number to the API Gateway. 3. The Gateway forwards the request to the Orchestrator, which fans out to the Agent Services as in UC-1. 4. The Orchestrator stores the aggregated report for the Dashboard. 5. The Dashboard renders the report grouped by agent and severity. 6. The Check Run and inline comments are posted to the pull request as in UC-1.
Alternate / Exception Flow	If the pull request is closed, reviews are off, or a review started in the last 30 s, the Dashboard says why and no review is run.
Postconditions	The Dashboard shows the new review of the pull request's head commit; nothing is written to the pull request branch.

UC-3 — Apply an AI-Suggested Fix

Field	Description
Actors	PR Author (primary), GitHub / Web Dashboard (client), API Gateway
Preconditions	A review report containing at least one high-confidence suggested change exists.
Main Flow	1. PR Author opens the suggested-change comment on GitHub, or the finding card in the Dashboard. 2. PR Author reviews the proposed diff. 3. PR Author clicks “Commit suggestion” (GitHub) or “Apply fix” (Dashboard). 4. The client commits the change to the PR branch.
Alternate / Exception Flow	PR Author clicks “Reject” or takes no action — the suggestion remains visible but no code is changed.
Postconditions	The accepted change is present in the PR branch; the finding is marked “resolved” on the next review run.

UC-4 — Configure Repository & Agent Settings

Field	Description
Actors	Repository Admin (primary), Web Dashboard, API Gateway
Preconditions	The admin is authenticated and the repository is registered under their GitHub installation.
Main Flow	1. Admin opens Agent Config for a repository from the Web Dashboard. 2. Admin enables/disables individual agents (required, FR-WEB-02, Must).

Field	Description
	3. Admin optionally overrides the platform-wide default confidence threshold of 80%/"High" (FR-WEB-03, Should — a Phase 2 enhancement; if the admin does not act, the default from FR-ORC-06 continues to apply). 4. Admin clicks “Save changes”. 5. The Dashboard sends the updated configuration to the API Gateway, which persists it in PostgreSQL.
Alternate / Exception Flow	If the save request fails validation (e.g. no agents enabled), the Dashboard shows an inline error and does not persist the change. If the threshold-override control (step 3) has not shipped yet, the admin can still complete steps 1–2, 4–5 using only the platform default threshold.
Postconditions	Subsequent review jobs for the repository use the updated agent selection and confidence threshold (or the platform default, if no override has been set).

UC-5 — View Review History & Combined Report

Field	Description
Actors	Tech Lead / Repository Admin (primary), Web Dashboard, API Gateway
Preconditions	At least one review has completed for a repository the user has access to.
Main Flow	1. User opens “Review History” on the Web Dashboard. 2. The Dashboard requests the review list from the API Gateway, filtered by repository, status, severity and date. 3. User selects a specific pull request to open its combined report detail view. 4. The Dashboard displays findings from all five agents for that PR, filterable by agent.
Alternate / Exception Flow	If no reviews match the selected filters, the Dashboard shows an empty state instead of a table.
Postconditions	The user has visibility into historical and current review outcomes across their repositories.

6. Interface Requirements

The diagram below gives a system-context view of every interface Code-Sentinel exposes or consumes, grouped into the three categories detailed in Sections 6.1–6.3: external third-party interfaces, internal service-to-service calls, and the human-facing UI surfaces. No agent service calls another agent directly — every hand-off routes through the Orchestrator.

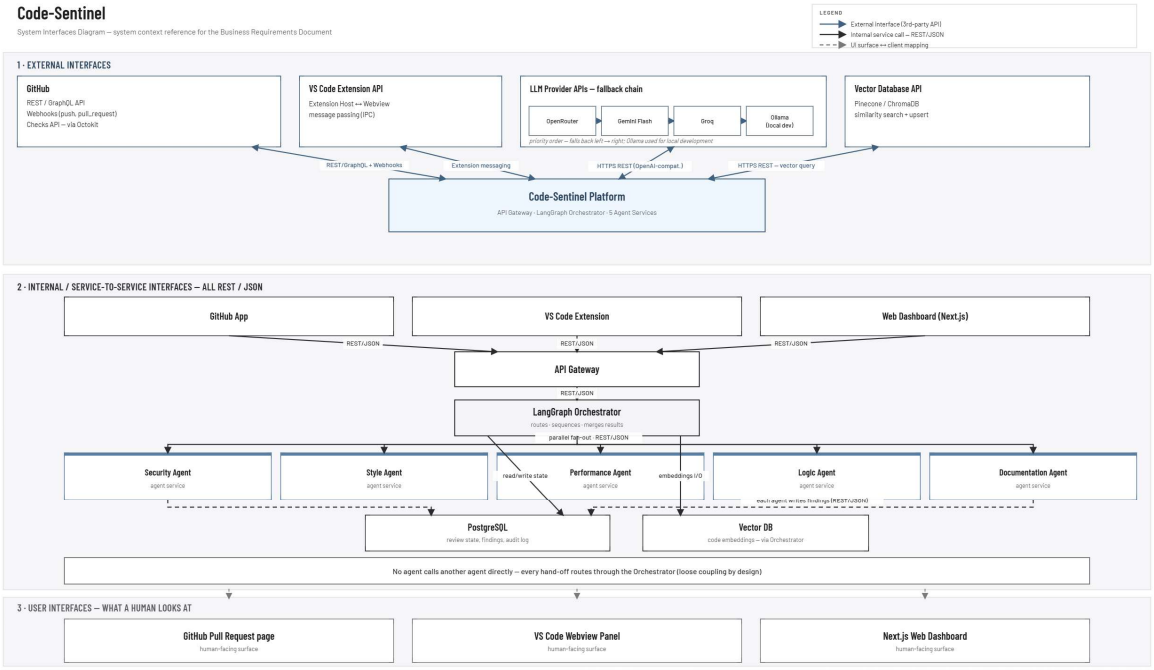


Figure 6.1 — Code-Sentinel system interfaces diagram

6.1 External Interfaces

Interface	Purpose
GitHub REST/GraphQL API, Webhooks & Checks API (via Octokit)	Receive pull_request webhook events; post Check Runs, review comments and suggested changes.
Supabase Auth API	Email, GitHub and Google sign-in, sessions and TOTP for the Dashboard; the Gateway verifies its access tokens.
LLM Provider APIs — OpenRouter, Gemini Flash, Groq (fallback chain), Ollama (local dev)	Power the Security, Logic, Performance and Documentation agents' AI analysis via a single provider-agnostic abstraction layer.
Vector Database API — Pinecone / ChromaDB	Store and query embeddings of previously flagged issues for context-aware review.

6.2 Internal (Service-to-Service) Interfaces

All internal interfaces are REST/JSON, consistent with the project's SOA design: no agent service calls another agent directly.

From → To	Contract
Client (GitHub App / Dashboard) → API Gateway	HTTPS REST/JSON, authenticated via GitHub App installation token or session.
API Gateway → Orchestrator	Internal REST/JSON review-request contract: repository, PR/file context, diff, enabled agents, confidence threshold.
Orchestrator → each Agent Service	POST /review {diff, file context, language} → {findings[], severity, confidence} per agent, called independently and in parallel.
Orchestrator / Agents → PostgreSQL	Persist reviews, users, and GitHub installation records.
Orchestrator → Vector DB	Write embeddings for new findings; query for semantically similar past issues.

6.3 User Interfaces

Surface	Description
GitHub Pull Request page	Rendered by GitHub itself from the Checks/Review API payloads Code-Sentinel posts: a Check Run summary plus inline suggested-change comments.
Dashboard Review Page	The page for one review showing the combined report grouped by agent and severity, with Re-run review and accept/reject actions.
Next.js Web Dashboard	Review history, per-repository agent configuration, and combined report detail views; responsive down to mobile widths.

6.4 Communication & Hardware Interfaces

- All client-service and service-service traffic is carried over HTTPS with TLS 1.2 or higher.
- Linters and formatters invoked by the Style Agent run inside a resource-limited Docker sandbox container, isolated per request.
- GitHub webhook delivery uses HMAC-SHA256 signed payloads, verified by the API Gateway before processing.

7. Wireframes — UI/UX Mockups (Initial Draft)

The low-fidelity wireframes below are an initial draft of the two client surfaces described in Section 1.2 and the Interface Requirements in Section 6.3. They establish layout and information hierarchy only; visual design (typography, color system, spacing scale) will be finalized in the System Design Document.

W1 — GitHub Pull Request: Inline Suggested Change

Supports FR-GH-02, FR-GH-03, FR-GH-04 and UC-1/UC-3. The Check Run summary card gives an at-a-glance combined result; the inline comment shows the Security Agent's finding with a committable suggested diff.

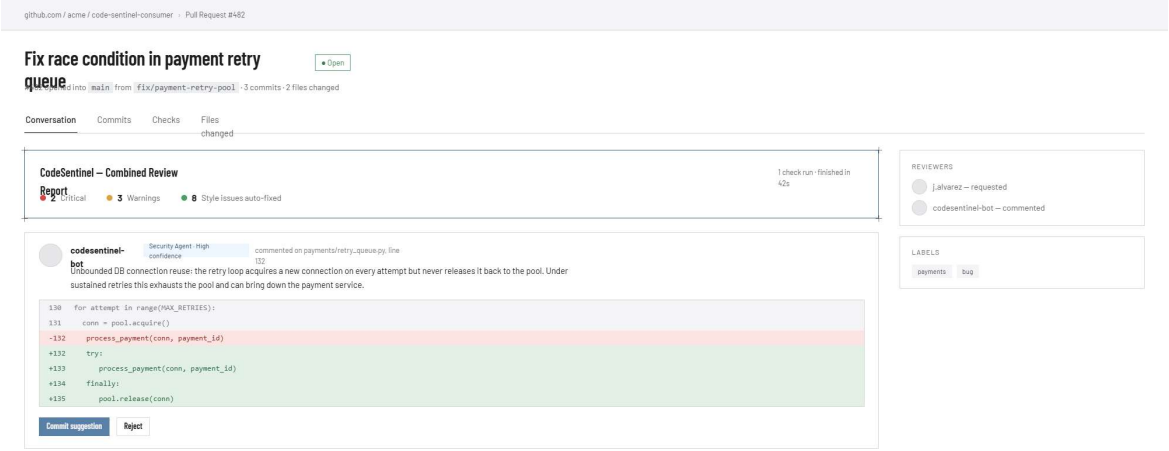


Figure 7.1 — GitHub PR inline suggested-change comment (initial draft)

W3 — Web Dashboard: Review History

Supports FR-WEB-01 and UC-5. Filters by repository, status, severity and date sit above a sortable table of past reviews with status chips and finding counts.

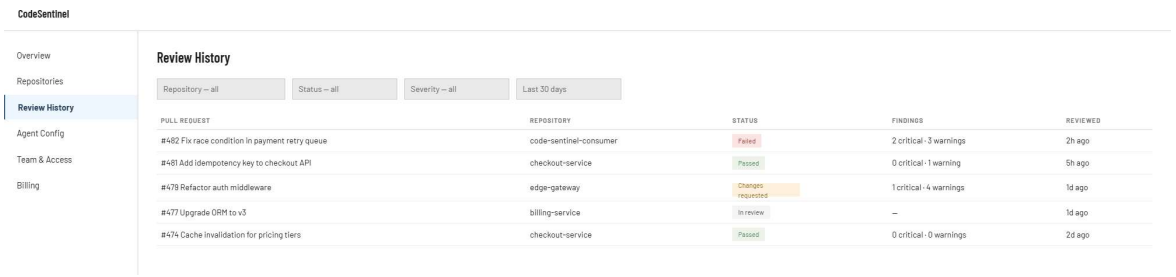


Figure 7.3 — Web Dashboard review history list (initial draft)

W4 — Web Dashboard: Repository & Agent Configuration

Supports FR-WEB-02, FR-WEB-03 and UC-4. Per-repository toggles let a Repository Admin enable/disable each of the five agents, and a slider sets the confidence threshold for auto-suggested fixes.

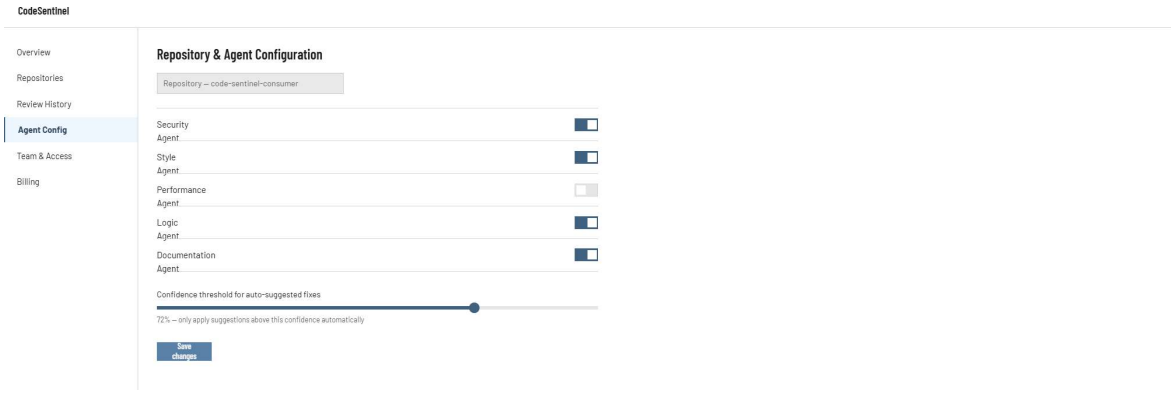


Figure 7.4 — Web Dashboard repository/agent configuration (initial draft)

W5 — Web Dashboard: Combined Review Report Detail

Supports FR-WEB-04, FR-ORC-03, FR-ORC-04 and UC-5. Findings from all five agents are merged into one list, de-duplicated, ranked by severity and filterable per agent — the visual equivalent of the GitHub Check Run summary, viewed from the dashboard.

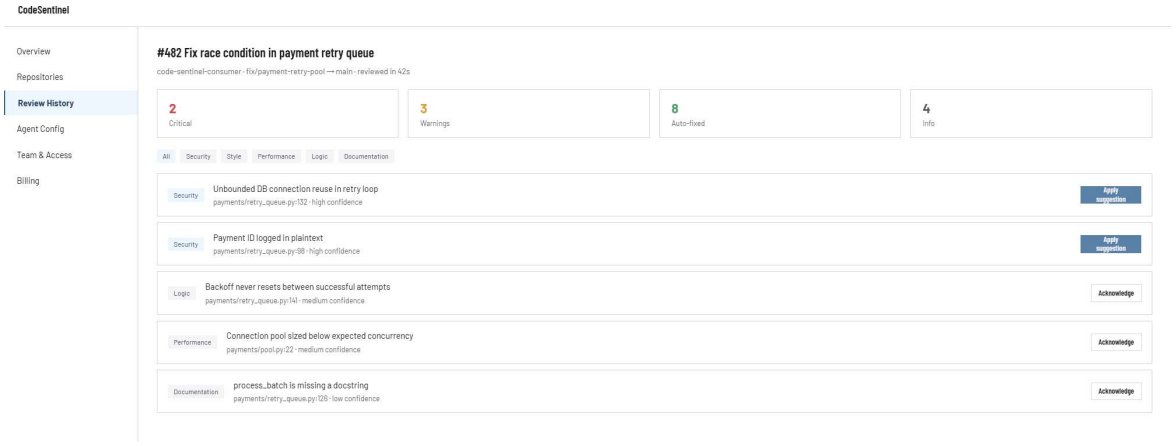


Figure 7.5 — Web Dashboard combined review report detail (initial draft)

W6 — Combined Review Report: High-Fidelity Vision Mockup

Distinct from the low-fidelity wireframes above, this mockup applies real visual design (color, typography, spacing) to the Combined Review Report screen — the product's core differentiator — to anchor the intended UI direction ahead of the System Design Document.

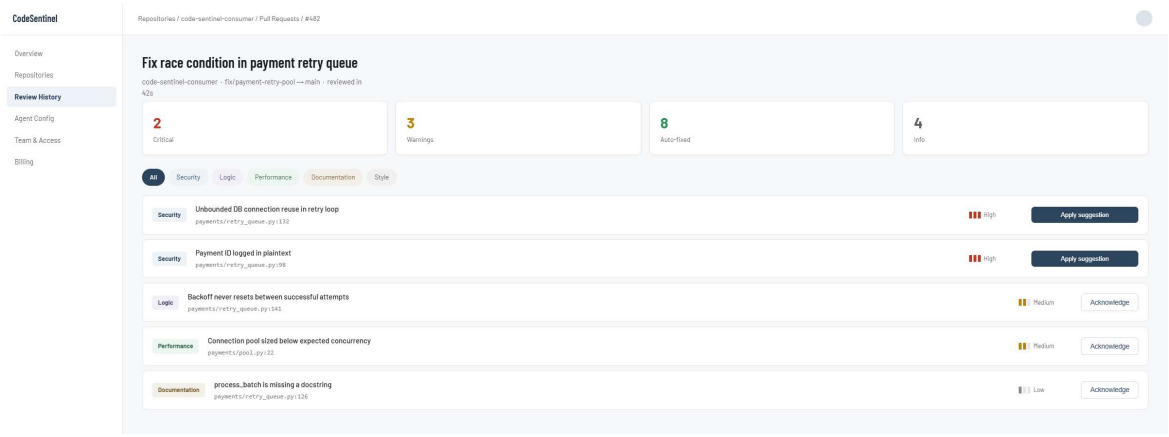


Figure 7.6 — Combined Review Report, high-fidelity vision mockup

8. Acceptance Criteria

Acceptance criteria are written in Given/When/Then form and map one-to-one to the user stories in Section 5.1. Priority mirrors the linked user story: Must-tier criteria gate the MVP milestone and must be demonstrable end-to-end; Should/Could-tier criteria are verified only once their underlying feature ships, and their Given clause states that dependency explicitly.

AC	US	Pri.	Given	When	Then
AC-01	US-01	**Must**	A GitHub repository has CodeSentinel installed.	A developer opens or pushes to a pull request.	A review job starts within 5 seconds and a GitHub Check Run appears showing “in progress”.
AC-02	US-02	**Must**	A review job has completed.	The developer opens the Check Run / PR comments.	Findings from all responding agents are shown in one combined report, grouped by severity, with no duplicate entries for the same issue.
AC-03	US-03	**Must**	A changed file has auto-fixable lint/style issues.	The Style Agent runs on the diff.	Fixes are applied via a suggested change without any LLM call, and remaining non-auto-fixable issues are reported separately.
AC-04	US-04	**Must**	A high-confidence AI-suggested fix is posted as a PR comment.	The developer clicks “Commit suggestion” (or “Reject” / takes no action).	The change is applied to the PR branch only on “Commit suggestion”; otherwise no code is changed.
AC-05	US-05	**Must**	The developer is signed in and the pull request is open.	The developer clicks “Re-run review” on the pull request in the dashboard.	The review page opens and shows results within the configured per-agent timeout.
AC-06	US-06	**Must**	A PR contains only style/formatting issues.	CodeSentinel completes its review.	The Check Run passes without requiring the Tech Lead to comment on formatting.
AC-07	US-07	**Must**	The Agent Config page is open for a repository.	An admin toggles an agent off and saves.	That agent is excluded from every subsequent review job for that repository.
AC-08	US-08	**Should**	The Agent Config page is open for a repository and the threshold-override control (FR-WEB-03) has shipped.	An admin sets the confidence threshold to “High (80%)” and saves.	Only findings with confidence ≥ 80% are posted as suggested changes; lower-confidence findings still appear in the report without an auto-suggested fix. Until this control ships, FR-ORC-06's platform default (80%) applies and this AC is not yet verifiable.
AC-09	US-09	**Must**	At least one review has completed.	The admin/reviewer opens Review History.	All reviewed PRs across installed repositories are listed with status, finding counts and timestamps, filterable by repository, status, severity and date.

AC	US	Pri.	Given	When	Then
AC-10	US-10	**Should**	A pull request is opened by a first-time or external contributor.	The webhook event is received.	CodeSentinel runs the same automatic review as for internal contributors, with no special action required by the maintainer.
AC-11	US-11	**Must**	A solo developer with no assigned human reviewer has CodeSentinel installed.	They open a PR in their own repository.	CodeSentinel still runs and posts a combined review report as the sole review gate.
AC-12	US-12	**Must**	One agent service is unavailable or exceeds its timeout.	The Orchestrator aggregates results.	The combined report is still produced from the remaining agents; the unavailable agent is marked “no result” rather than failing the whole review.
AC-13	US-13	**Could**	The vector database (FR-VDB-01/02) is implemented and a new finding matches a previously resolved issue above the similarity threshold.	The report is generated.	The finding includes a reference to the similar past issue and its resolution. This AC applies only once the Could-priority vector-DB feature ships.

9. Assumptions and Constraints

9.1 Assumptions

- Repositories under review use GitHub as their version control host; other Git hosts (GitLab, Bitbucket) are not targeted this cycle.
- Target codebases are primarily JavaScript/TypeScript or Python, matching the linters and analysis agents built for the MVP.
- Developers have or can obtain API access to at least one of the configured LLM providers, or use local Ollama during development.
- Reviewers/admins interacting with the dashboard have a modern evergreen browser (Chrome, Edge, Firefox).

9.2 Constraints

- The 14-week project timeline (27-Jul-2026 to 22-Nov-2026) fixes the delivery window; the MVP must be ready by Week 10.
- LLM usage during development is constrained to free/low-cost tiers via the OpenRouter → Gemini Flash → Groq → Ollama fallback chain.
- Style/lint auto-fixes must remain deterministic and LLM-free so they can be applied without developer review (NFR-07).
- No agent service may call another agent directly — all coordination goes through the Orchestrator, to preserve SOA loose coupling.

10. Document Approval

This Business Requirements Document was prepared by Durgesh, incorporating requirement study and API contract drafting carried out collaboratively with the rest of Team Code-Sentinel. This revision addresses internal review feedback on requirement traceability and priority consistency; no scope changes were introduced.

Name	Enrollment No.	Role & Responsibility
Sumit Goyal	202512092	Orchestrator (LangGraph) — parallel fan-out, timeout handling, LLM abstraction layer
Durgesh	202512051	Orchestrator — result aggregation, de-duplication, severity ranking; Vector DB embeddings & similarity search
Tej Prakash Tak	202512113	Security Agent (SAST, secret detection) and Logic Agent (bugs, edge cases)
Nevil Nandasana	202512009	Style Agent — language dispatcher, Docker sandbox, linter integration
Parin Makwana	202512098	Performance Agent and Documentation Agent (docstring suggestions)
Ayush Soni	202512030	API Gateway (Express + TypeScript) — auth, routing, webhook receipt; PostgreSQL schema
Vatsal Kamani	202512068	GitHub App — webhooks, Octokit, Checks & Review API; Docker and CI deployment
Moksh Mehta	202512120	Next.js dashboard, review history & reports